

Report - Version 7: LLM Router

Introduction

Version 7 introduces the LLM Router.

In the previous version, the project introduced the Tool Registry. The agent already had a central source for tool functions, parameters, descriptions, and synonyms, while the Router still selected tools through manually defined routing rules.

Version 7 keeps the same architecture, but changes how the Router understands the user request. Instead of scoring manually defined synonyms, the Router now uses a small pretrained language model to select the most appropriate registered tool.

The language model runs locally inside the notebook. It is downloaded from Hugging Face and does not use an external inference API.

Goal of Version 7

The main goal of Version 7 is to introduce language-model-based tool routing while keeping the rest of the agent unchanged.

The Router should use the names and descriptions stored in the Tool Registry to understand which tool can handle the request. If no registered tool is appropriate, it should return no tool.

Only the Router uses an LLM in this version. The Planner remains rule-based, the Generic Parser remains rule-based, and execution and memory management continue to work as before.

What changes compared to Version 6

Version 6 centralized tool management through the Tool Registry. The Router still depended on registered synonyms, scores, and priorities to select a tool.

Version 7 removes that rule-based selection logic from the Router. The Tool Registry remains unchanged, but the Router now reads tool names and descriptions and presents them to a local language model.

The synonyms are still stored in the registry for continuity with Version 6, but the Version 7 Router no longer uses them to make the routing decision.

New concept: LLM Router

The LLM Router converts tool selection into a small language-understanding task. The user request and the available tool descriptions are placed in a routing prompt, and the model is asked to return one allowed tool name or none.

Version 7 uses the following local model:

```
MODEL_NAME = "Qwen/Qwen2.5-0.5B-Instruct"
```

The model and tokenizer are loaded with Transformers. The runtime uses CUDA when available and otherwise falls back to the CPU:

```
device = "cuda" if torch.cuda.is_available() else "cpu"
```

```
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model = AutoModelForCausalLM.from_pretrained(MODEL_NAME).to(device)
model.eval()
```

The model is used locally after loading. Hugging Face is used to obtain the pretrained model files, not as an external inference service.

How the LLM Router uses the Tool Registry

The Tool Registry remains the shared source of tool information. For routing, Version 7 builds a compact list containing each registered tool name and its description.

```
for tool_name, tool_info in tool_registry.items():
    available_tools.append(
        f"- {tool_name}: {tool_info['description']}"
    )
```

The Router prompt also lists the allowed answers and explicitly tells the model to return none when no tool can directly handle the request.

The model generates a short deterministic response with sampling disabled. The Router then validates the generated text against the registered tool names.

```
outputs = model.generate(
    **inputs,
    max_new_tokens=8,
    do_sample=False
)

for tool_name in tool_registry:
    if re.search(rf"\b{re.escape(tool_name)}\b", response):
        return tool_name

return None
```

This final check does not decide the intent. The language model makes the semantic routing decision; the Python code only converts the generated answer into a valid registered tool name.

Complete architecture in Version 7

The execution flow is now:

```
Request -> LLM Router -> Planner -> Generic Parser -> Executor -> Memory Manager -> Memory
                                                |
                                                v
                                                Output
```

Component responsibilities

The responsibilities in Version 7 are:

Tool Registry = stores tools and their metadata

LLM Router = selects the tool semantically using the local language model

Planner = creates the execution plan using registered metadata

Generic Parser = extracts the arguments required by the selected tool

Executor = validates and executes the registered function

Memory Manager = saves, reads, and clears memory

Agent = coordinates

Memory = stores the state

The Agent remains the coordinator. The new intelligence is intentionally limited to the routing step so that the architectural change remains easy to understand.

Agent state in Version 7

The agent state keeps the same structure used in the previous versions.

Introducing an LLM Router does not require a new state field.

```
{
  "request": "Could you calculate the product of 2 and 3?",
  "plan": {
    "tool": "multiplication",
    "reason": "Multiply two numbers.",
    "needs_arguments": ["a", "b"]
  },
  "action": "multiplication",
  "arguments": {"a": 2, "b": 3},
  "result": 6
}
```

The difference is how action is selected: the Router now interprets the request with the local model before the Planner creates the plan.

Unsupported requests

Unsupported requests are now evaluated semantically instead of through synonym matching.

For example:

```
agent("What is the weather in Rome?")
```

There is no weather tool in the registry. The LLM Router therefore returns no tool, the Planner creates a plan with no selected action, and the Agent returns the existing fallback result.

The interaction is still saved through the Memory Manager exactly as in Version 6.

Tests

The tests verify both compatibility with the previous architecture and the new semantic routing behavior:

```
clear_memory()
tool_registry

agent("What is 2 + 3?")
agent("Hello, what is 2 + 3?")
agent("Hello, my name is luca.")
agent("What is the weather in Rome?")
agent("Could you calculate the product of 2 and 3?")

get_memory()
get_last_interaction()
```

These tests verify that:

- the LLM Router selects addition for a direct arithmetic request;
- the LLM Router identifies addition as the main intent even when the request starts with a greeting;
- the greeting tool still works with the existing rule-based name extraction;
- an unsupported weather request returns no tool;
- the word product is understood as multiplication even though it is not one of the multiplication routing synonyms from Version 6;
- the Planner, Generic Parser, Executor, Memory Manager, and Agent continue to work without architectural changes;
- memory still stores the full state of every interaction.

What Version 7 teaches

Version 7 teaches the first practical use of a language model inside the agent architecture:

semantic routing can replace manually maintained intent rules

The Router no longer needs a synonym score for every way a user may express the same intent. Tool descriptions become useful machine-readable context for selecting an action.

At the same time, the version keeps the LLM responsibility narrow. The model selects the tool, but it does not yet extract arguments, create multi-step plans, execute functions, or manage memory.

Limitations

Version 7 introduces an LLM, but several limitations remain:

- the small local model can still make routing mistakes;
- the Planner remains rule-based;
- the Generic Parser still uses manual extraction;
- the Tool Registry remains in-memory and tool registration is manual;
- memory remains a simple list;
- the agent still performs one action at a time.

Conclusion

Version 7 is a natural evolution of Version 6.

The Tool Registry remains the central source of tool information, but routing is no longer based on manually defined synonym scores. A local pretrained language model now interprets the user request and selects the registered tool that can handle it.

The rest of the architecture remains deliberately unchanged: the Planner is still rule-based, the Generic Parser still extracts arguments with explicit rules, the Executor runs the registered function, and the Memory Manager stores interaction state.

The successful product test demonstrates the main benefit of the new Router: it can recognize a semantic intent that was not explicitly encoded as a routing synonym. The unsupported weather test also shows that the Router can return no tool when the registry does not contain an appropriate capability.

The next step will be Version 8, where the project will introduce an LLM Parser and move argument extraction from manual rules to language-model-based parsing.